

Finex

A Financial DSL



Team

Name	Roll Number	Role
Sammathamu Vandana Sahani	cs23btech11054	Project Manager
Kukkala Aashritha Reddy	cs23btech11030	System Architect
Nishi Baranwal	cs23btech11041	System Integrator
Anusha Kumaresan	cs23btech11006	Language Guru
Paidala Vindhya	cs23btech11044	Language Guru
Yoligi Harshini	cs23btech11065	Tester
Gopu Bala Anthony Poojitha	cs23btech11020	Tester

Contents

1	Introduction	3
1.1	What is Finex?	3
1.2	Current Challenges in the Financial Domain	3
1.3	The Solution: How Finex Solves the Problem ?	4
2	Motivations and Target Users	4
2.1	Target Demographic	4
2.2	Challenges addressed	4
3	Design Principles	5
4	Language Features	5
4.1	Introduction	5
4.2	Lexical Conventions	5
4.2.1	Tokens	5
4.2.2	Comments	5
4.2.3	Identifiers	6
4.2.4	Keywords	6
4.2.5	Constants and Literals	7
4.2.6	Macros	7
4.2.7	Delimiters	8
4.2.8	Scoping	8
4.3	Data Types	8
4.3.1	Built-in Primitive Types	8
4.3.2	Financial Primitives	9
4.3.3	User-defined Types	9
4.3.4	Qualifiers and Storage Class Specifiers	9
4.3.5	Type Conversions	9
4.4	Operators	10
4.5	Statements	10
4.5.1	Expression statements	10
4.5.2	Compound Statements	10
4.5.3	Conditional / Selection Statements	10
4.5.4	Iteration Statements	11
4.5.5	Jump Statements	11
4.6	Exception Handling and Assertion Statements	11
4.7	Input - Output	12
4.8	Functions	12
4.8.1	Declarations	12
4.8.2	Definitions	12
4.8.3	Function call	13
4.9	Libraries	13
4.9.1	Accounting	13
4.9.2	Containers	14
4.9.3	Datastream	16
4.9.4	Statmodel	16
4.9.5	Importing and Accessing Libraries	17
4.10	File extensions	18

5	Technical Advantages	18
6	Comparison with Alternatives	19
7	Programming Principles	19
8	Code samples	20
9	Precedence and Associativity	22

1 Introduction

1.1 What is Finex?

Finex is a domain-specific language (DSL) designed specifically for quantitative finance. It provides a single, unified framework where finance professionals can simply express what they want to compute, instead of worrying about how to implement it.

Finex supports the complete financial workflow, including portfolio management for tracking and analyzing investments, and risk analysis for measuring exposures under different market conditions. It provides derivative pricing for instruments such as options and futures, and time series forecasting for predicting market trends. The language also enables foreign exchange trading and algorithmic trading through automation and rule-based execution. In addition, it supports fraud detection and core banking operations like account management, deposits, lending, and payments. Finex further integrates statistical methods, machine learning models, and automated financial reporting with visualization. In short, Finex serves as both a computational engine and an abstraction layer: professionals state what they want to achieve, and the system determines the most effective way to perform it.

1.2 Current Challenges in the Financial Domain

The financial industry faces several deep-rooted challenges that slow down innovation, increase costs, and create operational risks. Some of the most critical issues are:

- **Tool Fragmentation** : Today's financial workflows are spread across multiple tools—Python for backtesting, C++ libraries for pricing, Excel for accounting, and separate software for reporting. This fragmentation forces professionals to constantly switch between platforms and manually integrate results together. The result is inefficiency, higher error risk, and difficulty in maintaining a consistent workflow.
- **Expertise Gap** : Implementing modern financial strategies often requires mastering multiple programming languages, libraries, and analytical tools. This steep learning curve slows innovation, increases dependency on specialized teams, and limits accessibility for smaller firms and professionals without extensive technical training.
- **Complexity of Derivative Pricing and Risk Management** : Valuing derivatives often requires advanced methods like Monte Carlo or PDE solvers. Today, professionals must manually pick, code, and verify these methods, making the process slow, error-prone, and hard to scale.
- **Scalability and Large Time Series Data** : Modern financial analysis relies on extensive historical data, including long time series for stocks, indices, and other instruments. Traditional tools struggle to process and analyze large datasets efficiently. Running simulations, backtests, or forecasting models on these time series can be slow and resource-intensive, making it difficult for smaller firms to perform sophisticated analysis.
- **Reporting and Data Visualization Challenges** : Generating meaningful reports often requires manual plotting and formatting of results. Visualizing portfolio performance, risk metrics, or time series statistics can be cumbersome across different tools.

1.3 The Solution: How Finex Solves the Problem ?

- **Unified Platform** : Finex combines portfolio management, derivative pricing, backtesting, accounting, and reporting in a single language. Professionals no longer need to switch between multiple platforms, reducing errors and improving workflow efficiency.
- **Declarative Simplicity** : Finex lets users specify what they want to compute while the engine handles how to do it. This reduces the need for proficient programming skills, making complex analyses accessible to a more diverse set of professionals.
- **Automated Derivative Pricing** : Users define derivatives naturally, and Finex automatically selects the correct pricing method (Monte Carlo, PDE, etc.). This removes manual coding, minimizes errors, and makes sophisticated risk management easier.
- **Scalable Time Series Analysis** : Finex efficiently processes large historical datasets, enabling fast simulations, forecasts, and backtests. Smaller firms can perform advanced analyses without heavy custom infrastructure.
- **Integrated Reporting and Visualization** : Finex generates graphs, charts, and statistical summaries directly from analysis. This streamlines reporting, ensures consistency, and eliminates the need for manual plotting.

2 Motivations and Target Users

2.1 Target Demographic

Finex is designed to empower a wide range of finance professionals by streamlining complex quantitative finance analysis and workflows through an intuitive, domain-specific language. Portfolio managers and traders benefit from simplified portfolio construction, rebalancing, and risk analysis commands that reduce operational overhead and increase precision. Quantitative analysts and risk managers can perform statistical assessments and scenario-based stress tests without the burden of verbose coding, enabling fast and reliable risk evaluation. Financial engineers and developers leverage Finex to build, backtest, and deploy algorithmic trading strategies that integrate advanced modeling and forecasting techniques seamlessly. Investment advisors and wealth managers use the language to generate comprehensive reports, visualize portfolio trends, and communicate strategies effectively to clients. Additionally, data scientists and machine learning practitioners can easily incorporate predictive models and fraud detection algorithms into trading workflows, accelerating innovation and improving decision-making accuracy. Together, these user groups benefit from a unified platform that enhances productivity, safety, and collaboration across the financial ecosystem.

2.2 Challenges addressed

Traditional financial workflows often depend on a tangled combination of Python backtesters, C++ pricing libraries, manual stress tests, spreadsheet macros, and ad-hoc scripts for automation and reporting. This fragmented environment leads to operational risk, increased maintenance burden, and barriers to innovation, especially as data volumes and model complexity grow.

Traditional financial workflows often depend on a tangled combination of Python backtesters, C++ pricing libraries, manual stress tests, spreadsheet macros, and ad-hoc scripts for automation and reporting. This fragmented environment leads to operational risk, increased

maintenance burden, and barriers to innovation, especially as data volumes and model complexity grow.

3 Design Principles

The core design principles of Finex center on domain expressiveness, simplicity, safety, rich tooling, and performance. All critical financial constructs—ranging from assets (stocks, options, futures, cryptocurrency, accounts) and portfolios, to analytical constructs (time series, risk models, ML forecasts)—are first-class language types. This ensures that users are freed from boilerplate and can focus on their analytical logic rather than engineering and data plumbing. Simplicity and learnability are promoted through intuitive, declarative syntax with powerful default behaviors for common analytics, reporting, and trading tasks. Safety is embedded by design: the DSL employs a strong type system to prevent logical errors such as mismatched currencies or ill-formed portfolios and includes runtime validation for invariants like weights summing to unity or type-correct asset operations. Powerful features such as live trading, data access, and model training are permissioned, supporting robust access controls and authentication at the language level.

Tooling and interoperability are key strengths. Finex integrates automatically with real-time and historical market data sources, connects with established C++/Python model libraries for advanced pricing and forecasting, and provides seamless export to all standard reporting and visualization formats (CSV, PDF, HTML, dashboards). The runtime is optimized for performance and efficiency: back-end optimizations ensuring that high-frequency and large-scale scenarios run safely and quickly without user intervention.

Above all, Finex carefully balances human readability with machine-level optimization. The DSL is designed so that programs are readily understandable and auditable by financial professionals, yet, under the hood, benefit from aggressive compilation and runtime strategies for maximal throughput and reliability. In doing so, Finex dramatically reduces operational complexity, helps enforce safety and correctness, and enables innovation for modern quantitative finance—all within a single, unified, and well-governed platform.

4 Language Features

4.1 Introduction

Finex is a case-sensitive, declarative-first language designed for financial applications. It combines clear syntax rules, built-in financial data types, and safe design choices like no pointers or manual memory management. The goal is to keep financial coding precise, readable, and reliable while directly supporting domain needs.

4.2 Lexical Conventions

4.2.1 Tokens

Tokens such as identifiers, keywords, constants, string literals, operators, and other separators exist.

4.2.2 Comments

The DSL supports two types of comments:

- **Single-line comments:** Begin with `##` and continue until the end of the line.
- **Multi-line comments:** Enclosed between `/#` and `#/`, and may span multiple lines.

Single-line comment example:

```
1      ## This is a single-line comment
2      balance = 1000 ;  ## This sets the initial balance
```

Multi-line comment example:

```
1      /#
2          This is a multi-line comment.
3          It can span multiple lines and is ignored by the parser
4          .
5      #/
total = income - expenses ;
```

Comments are ignored during execution. They cannot nest or be placed within strings.

4.2.3 Identifiers

In the DSL, identifiers are used to name variables, functions, constants, and other user-defined entities. Identifiers are composed of letters, digits, and the underscore character ('_').

An identifier must begin with a letter (either lowercase or uppercase) or an underscore, and may be followed by any combination of letters, digits, or underscores.

$$\text{Identifier} := [\text{a-zA-Z_}][\text{a-zA-Z_0-9}]^*$$

Identifiers are **case-sensitive**, meaning that `totalAmount` and `TotalAmount` are considered distinct.

Reserved keywords in the DSL cannot be used as identifiers. Variables and functions need to be declared before use.

- Declaration - just defines the variable with a type and name.
- Initialization - defines the variable and assigns a value at the same time.

```
1      int counter ;
2      real price = 105.25 ;
```

4.2.4 Keywords

Keywords are reserved words in the DSL that have special meaning and cannot be used as identifiers. The language is **case-sensitive**, so `Amount` and `amount` are treated as distinct.

The following are recognized as keywords:

- **Built-in Data Types:**

- Primitive types: `int`, `real`, `char`, `string`, `bool`, `datetime`
- Finance specific types: `amount`
- User defined types: `label`, `record`

- **Logical Operators:**

- and, or, not
- **Membership Operator:**
 - in
- **Constants / Literals:**
 - true, false (boolean literals)
- **General**
 - if, else, for, while, break, continue, try, catch, throw, check, from, to, step, modifiable, default, as, return, void, const, nonneg, extern, static, using

Note: Attempting to use any keyword as an identifier will result in a syntax error. All keywords must be written in lowercase unless specified otherwise by future language extensions.

4.2.5 Constants and Literals

- Integer: 123, 9999
- Real: 1.23, 99.99
- Amount: 100.50USD, 5000INR, 22.50EUR
- Character: 'a', 'z', '\n' (newline character)
- String: "AAPL", "Hello, World!", "NASDAQ"
- Boolean: true, false
- Enumeration constant:
 - label keyword creates a set of named constants which are of type int. They can be referred to by name.
 - Declaration: label Currency = { USD, EUR, JPY }
 - Usage: Currency::USD, Currency::JPY
- Datetime: {2025-09-10, 16:00:00}

4.2.6 Macros

Macros allow defining symbolic constants or reusable values that get replaced during compilation.

```

1      define SIZE as 10 ;
2      print("Array size is", SIZE) ;

```


4.2.7 Delimiters

- Curly Braces: `{}` - group multiple statements or record fields
- Square Brackets: `[]` - define lists (similar to vector)
- Parentheses: `()` - enclose expressions or function calls
- Angle Brackets: `<>` - specify container types
- Double Quotes: `"` - enclose string literals
- Single Quotes: `'` - enclose single characters
- Semi Colon: `;` - statement terminator
- Comma: `,` - separates elements in a list or map
- Colon: `:` - separates key and value in maps
- Space: - separates tokens like keywords, types, and identifiers
- Arrow: `->` - access methods of objects
- Double Colon: `::` - accessing enum values
- At: `@` - accessing library

4.2.8 Scoping

Finex allows both local and global scopes. Entities with global scope can be accessible anywhere below their declaration in the code file. Entities with local scope can be accessible anywhere below their declaration in the block they are present. Block scope is restricted by `{` and `}`.

4.3 Data Types

4.3.1 Built-in Primitive Types

- **int** – A standard signed integer type, used for counters, identifiers, or loop indices. The system provides automatic type promotion to larger internal representations when values exceed the standard range, thereby eliminating overflow concerns.
- **real** - A numeric type representing fixed-point decimal values, designed for precise financial computations such as monetary amounts, interest rates, and ratios. Internally, values are stored as scaled integers (with fixed decimal precision) to ensure exact arithmetic and prevent rounding errors inherent in floating-point representations.
- **char** – Holds a single character, suitable for short labels or one-letter codes.
- **string** – A series of characters, typically used for names, tickers, or other text fields.
- **bool** – Represents true or false values. Useful for flags, conditional logic, and filters.
- **datetime** – Stores date and time information, such as timestamps, trade dates, or expiry dates. Ex:

```
1      datetime tradeTime = {2025-09-10, 16:00:00} ;
```

4.3.2 Financial Primitives

- **amount** – A composite type (**record**) that pairs a real value with a **currency** label. For example, 22.5USD is syntactic sugar that the system interprets as a real value 22.5 together with the currency USD. This type ensures precise financial calculations while keeping the syntax simple and readable.

Ex :

```
1          amount myMoney = 22.5USD ;
```

4.3.3 User-defined Types

- **label** - A user-defined enumerated type (enum) that represents a fixed set of named values. It is useful for things like categories, types, or currencies.

Ex :

```
1          label Currency = { USD, EUR, JPY } ;  
2          label OptionType = { Call, Put } ;
```

- **record** - A composite user-defined type (similar to a struct or a class with all fields public) that groups multiple values together. Records can include other records or primitive types, making them useful for representing complex objects like trades, accounts, or portfolios.

Ex :

```
1          record Trade = {  
2              string ticker ;  
3              amount price ;  
4              int quantity ;  
5          } ;  
6          record Portfolio = {  
7              list<Trade> trades ;  
8              amount totalValue ;  
9          } ;
```

4.3.4 Qualifiers and Storage Class Specifiers

- **Type qualifiers** - const, nonneg (type qualifier for non negative values)
- **Storage class specifiers** - extern, static, using ... as ... (using x as y)

4.3.5 Type Conversions

- **Implicit Conversion** : The system automatically converts between int and real, issuing a warning when precision may be lost (e.g., real to int). For financial amounts, operands in different currencies are normalized to a base currency (USD) before arithmetic operations. Ex: 10USD + 13.4INR is normalized to USD currency.
- **Explicit Conversion** : Users can enforce type conversion using casts, such as (int) price or (real) count. This ensures full control, especially when truncating decimals or overriding the default base currency conversion.

4.4 Operators

- Arithmetic: +, -, *, / , // (int division), %, ^
- Logical: and, or, not
- Unary: +, -
- Relational: >, <, >=, <=
- Equality : ==, !=, === (for type checking)
- Assignment: =, +=, -=, *=, /=, %=, ^=
- Comma: ,
- Domain-specific:
 - Key-value separator (colon) :
 - For accessing members of an object ->
 - For checking membership “in”
eg. if("AAPL" in myStocks)

4.5 Statements

4.5.1 Expression statements

Expression statements perform computations or actions such as assignments, arithmetic, or function calls, and are terminated by ‘;’.

4.5.2 Compound Statements

A compound statement (block) groups multiple declarations and statements inside `{ }`, so they can be used where a single statement is expected. Variables declared inside a block are local to it, and if they share the same name as an outer variable, the outer one is temporarily hidden.

4.5.3 Conditional / Selection Statements

- **if:**

```
1         if(condition){
2             code
3         }
```

- **if ... else:** In this DSL, an else is always associated with the nearest preceding unmatched if, following standard programming language convention.

```
1         if(condition){
2             code
3         }
4         else{
5             code
6         }
```

- **if ... else ... if ...:**

```

1      if(condition){
2          code
3      }
4      else if(condition){
5          code
6      }
7      ...

```

4.5.4 Iteration Statements

- **for:**

```

1      for(<variable> from <start_value> to <end_value> step <
        increment/decrement>){
2          code
3      }
4
5      ## for loop definition using membership operator 'in'
6      for(<variable> in <list/map/...>){
7          code
8      }

```

- **while:**

```

1      while(condition){
2          code
3      }

```

4.5.5 Jump Statements

- **break, continue:**

```

1      while(condition){
2          if(condition){
3              <code>
4              break;
5          }
6          if(condition){
7              <code>
8              continue;
9          }
10         <code>
11     }

```

- **return:** Exits from a function and optionally passes a value back to the caller.

4.6 Exception Handling and Assertion Statements

- **try - catch, throw:** try-catch is used for handling errors or exceptional cases, and throw is used to raise an exception.

```

1      try {
2          <code>
3          if (condition) {
4              throw <error> ;
5          }
6          <code>
7      }
8      catch (string error) {
9          print("Caught error: ", error) ;
10     }

```

- **check**: It used to enforce that a condition must hold true during execution; if it fails, the program stops with an error.

```

1      check (price > 0) ;    ## ensures no negative prices
2      check (quantity != 0) ;    ## prevents division by zero

```

4.7 Input - Output

- **scan** is used to take input from the user.
- **print** is used to display output.

```

1      int age = scan() ;
2      print("Hello, your age is ", age) ;

```

4.8 Functions

Functions in Finex allow reusable blocks of logic. A function must be declared before use, defined with its body, and can be called from anywhere in scope. The starting point of execution of Finex code is the `main` function.

4.8.1 Declarations

A function declaration specifies the function's **name**, **return type**, and **parameters** (with modifiers or default values) but does not include its body. All default parameters must appear on one side (typically at the end), allowing the compiler to enforce this rule during the semantic phase and expand missing arguments with defaults unambiguously.

Ex of parameters :

`real discountRate default 0.05` : A variable `discountRate` of type `real`, with a default value of 0.05

`amount cash modifiable` : A variable `cash` of type `amount` that is passed by reference, meaning any changes made inside the function will persist outside the function.

4.8.2 Definitions

A function definition provides the actual implementation (body) of the function.

4.8.3 Function call

A function call executes the function using arguments.

```
1      ## function declaration
2      int powerSum(int x, int y default 2) ;
3
4      ## function definition
5      int powerSum(int x, int y default 2) {
6          int temp = x^3 ;
7          return temp + y ;
8      }
9
10     ## function usage
11     int result1 = powerSum(5) ;      ## uses default y = 2
12     int result2 = powerSum(5, 10) ;  ## overrides y
```

4.9 Libraries

- **accounting** : portfolio, account, user, add_asset, remove_asset, curr_value, rebalance, scenario_analysis, sensitivity_analysis, create_account, deposit, withdraw, transfer, simple_interest, compound_interest, buy, sell, emi
- **containers** : list, map, set, multimap, multiset, stack, queue, dequeue, priority_queue (along with sort, find, count, swap, copy, move, replace, lower_bound, upper_bound, set_union, set_intersection, maximum, minimum, reverse functions)
- **datastream** : open_file(), close_file(), file, dataframe, (also time series), filter((duplicates, row/col, conditions)), substitute(), backtest
- **statmodel** : Monte_Carlo, Black_Scholes, mean, median, variance, stddev, volatility, pnl, var, cvar, sharperatio, max_drawdown, avg_drawdown, time_to_recovery, capital_gains, concentration, top_holdings, risk_contribution, price_derivative, binomial, trinomial, finite_difference, delta, gamma, vega, arima, garch, vector_ar, random_forst, xg.boost, lstm, compare_models, generate_report, plot_graph, rsi, sma, ema, macd

4.9.1 Accounting

accounting - the user is the main record representing a financial client or entity. Each user can have multiple accounts, and each account contains exactly one portfolio. Portfolios, in turn, hold various financial instruments represented as asset records. This hierarchy allows the language to model real-world finance structures in a natural and reusable way.

- **user** – The main record (represents a person or client). A user can have many accounts.
- **account** – Belongs to a user. Each account has one portfolio and money.
- **portfolio** – Holds all the investments of an account and cash.
- **asset** – It includes general type for all financial instruments which is an abstract class. Common details: currency, quantity, cash.
Different kinds of assets include: **cash, stock, etf, commodity, mutualFund, indexFund, bonds, options, futures, cryptocurrency, foreignExchange** which are derived classes of asset abstract class.

- **portfolio**: Represents a collection of assets held by an account.
- **account**: A financial entity that tracks balances, transactions, and holdings.
- **user**: Identifies an individual or entity managing accounts.
- **add_asset**: Adds a specified asset to a portfolio.
- **remove_asset**: Removes a specified asset from a portfolio.
- **curr_value**: Calculates the current total value of assets in the portfolio.
- **rebalance**: Adjusts asset allocations to meet target weights or constraints.
- **scenario_analysis**: Simulates portfolio performance under hypothetical market conditions.
- **sensitivity_analysis**: Measures how changes in inputs affect portfolio performance.
- **create_account**: Initializes a new financial account for a user.
- **deposit**: Adds funds to an account, increasing its cash balance.
- **withdraw**: Removes funds from an account, reducing its cash balance.
- **transfer**: Moves funds between two accounts.
- **simple_interest**: Calculates interest on a principal amount without compounding.
- **compound_interest**: Calculates interest with periodic compounding.
- **buy**: Executes a purchase of an asset, decreasing available funds.
- **sell**: Sells an asset, increasing available funds or balance.
- **emi**: Computes fixed equal monthly installments for loan repayments.

4.9.2 Containers

list (similar to vector), **map**, **multimap**, **set**, **multiset**, **stack**, **queue**, **dequeue**, **priority_queue** – Predefined container types to hold and organize multiple items, making it easy to insert, find, or sort elements.

Ex :

```

1      record Trade = {
2          string ticker ;
3          amount price ;
4          int quantity ;
5      } ;
6
7      ## list: store multiple trades
8      list<Trade> trades = [
9          { "AAPL", 150.25USD, 10 },
10         { "TSLA", 320.50USD, 5 }
11     ] ;
12
13     ## map: allocation percentages

```

```

14     map<string, real> allocs = { "Equity": 0.6, "Bonds": 0.3, "
    Commodities":
15     0.1 } ;
16
17     ## multimap: multiple trades per ticker
18     multimap<string, Trade> tradesByTicker ;
19
20     ## set: unique tickers
21     set<string> tickers = { "AAPL", "TSLA", "GOOG" } ;
22
23     ## multiset: multiple alerts of same type
24     multiset<string> alerts = { "MarginCall", "MarginCall", "
    PriceAlert" } ;
25
26     ## stack: pending orders
27     stack<Trade> pendingOrders ;
28
29     ## queue, dequeue, priority_queue
30     queue<string> notifications ;
31     dequeue<string> recentEvents ;
32     priority_queue<string> riskQueue ;

```

- **list**: A dynamic array of elements allowing indexed access.
- **map**: A collection of key-value pairs with unique keys.
- **set**: A collection of unique elements.
- **multimap**: A map that allows multiple values for the same key.
- **multiset**: A set that allows duplicate elements.
- **stack**: A last-in, first-out (LIFO) container with push/pop access.
- **queue**: A first-in, first-out (FIFO) container for ordered processing.
- **dequeue**: A double-ended queue allowing insertion/removal from both ends.
- **priority queue**: A queue where elements are ordered by priority.
- **sort**: Arranges elements in ascending or descending order.
- **find**: Searches for an element in a container.
- **count**: Counts occurrences of an element in a container.
- **swap**: Exchanges the contents of two containers or two elements.
- **copy**: Duplicates elements or containers to another location.
- **move**: Transfers ownership of elements without copying.
- **replace**: Substitutes occurrences of an element with another.
- **lower_bound**: Finds the first element not less than a given value.
- **upper_bound**: Finds the first element greater than a given value.

- **maximum**: Returns the largest element in a container.
- **minimum**: Returns the smallest element in a container.
- **reverse**: Reverses the order of elements in a container.

4.9.3 Datastream

- **open_file()**: Opens a data file for reading or processing.
- **close_file()**: Closes a previously opened file to release resources.
- **file**: Represents a raw data file.
- **dataframe**: A structured tabular format for working with rows and columns.
- **time_series**: A sequence of data points indexed by time, often used for financial analysis.
- **filter** : Filters data based on logical or numerical conditions.
- **substitute()**: Replaces missing or specified values in the dataset.
- **backtest**: Evaluates a strategy or model against past data to assess performance.

4.9.4 Statmodel

- **model**: It is a datatype used to define and work with statistical or mathematical models.
- **Monte_Carlo**: Simulates a wide range of outcomes using random sampling.
- **Black_Scholes**: Computes option pricing using the Black-Scholes formula.
- **mean**: Calculates the average value of a dataset.
- **median**: Finds the middle value in a sorted dataset.
- **variance**: Measures the dispersion of values from the mean.
- **stddev**: Computes the standard deviation of a dataset.
- **volatility**: Measures the standard deviation of returns over time.
- **pnl**: Calculates profit and loss from a trading strategy or asset.
- **var**: Estimates the Value-at-Risk for a portfolio at a given confidence level.
- **cvar**: Computes Conditional Value-at-Risk (expected shortfall beyond VaR).
- **sharpe_ratio**: Calculates risk-adjusted return using Sharpe Ratio.
- **max_drawdown**: Measures the largest peak-to-trough decline in value.
- **avg_drawdown**: Computes the average drawdown over a period.
- **time_to_recovery**: Time taken to recover from the last drawdown.
- **capital_gains**: Calculates gains from the appreciation of assets.

- **concentration**: Measures the degree of portfolio exposure to a small number of assets, indicating diversification levels.
- **top_holdings**: Lists the top assets in a portfolio by value or weight.
- **risk_contribution**: Quantifies the contribution of each asset to total portfolio risk.
- **price_derivative**: Computes the derivative of a price series.
- **binomial**: Prices options using the binomial tree method.
- **trinomial**: Prices options using the trinomial tree method.
- **finite_difference**: Solves PDEs for option pricing using finite difference methods.
- **delta**: Measures the sensitivity of option price to changes in the underlying asset.
- **gamma**: Measures the rate of change of delta with respect to the underlying asset.
- **vega**: Measures sensitivity of the option price to volatility changes.
- **arima**: Fits AutoRegressive Integrated Moving Average models to time series.
- **garch**: Fits Generalized Autoregressive Conditional Heteroskedasticity models.
- **vector_ar**: Models multivariate time series using Vector AutoRegression.
- **random_forest**: Trains an ensemble of decision trees for classification or regression.
- **xg_boost**: Applies gradient-boosted trees for efficient predictive modeling.
- **lstm**: Trains Long Short-Term Memory neural networks for sequential data.
- **compare_models**: Compares performance of multiple statistical or ML models.
- **generate_report**: Produces a summary report of model metrics, analysis and results.
- **plot_graph**: Visualizes data, models, or results using graphs.
- **rsi**: Computes Relative Strength Index for momentum analysis.
- **sma**: Calculates Simple Moving Average over a time window.
- **ema**: Calculates Exponential Moving Average with more recent weight.
- **macd**: Computes Moving Average Convergence Divergence (MACD) indicator.

4.9.5 Importing and Accessing Libraries

Libraries are imported using the `@` symbol, which allows programs to use built-in or user-defined types and features. The library imports also allow aliasing using `as`. Library features can be accessed with or without `@`, but in case of ambiguity (where error will be thrown by the compiler), `@` is required.

```

1      ## importing the library
2      @accounting ;
3      @containers as cont;
4
5      ## accessing the library with @
6      cont@list<Trade> trades ;
7      accounting@user ;
8
9      ## accessing the library without @
10     multimap<string, Trade> tradesByTicker ;

```

4.10 File extensions

In Finex, files are organized with .h for header files (declarations, constants, macros) and .fx for code files (program logic). External files can be included using @filename, which helps keep programs modular and reusable, similar to importing libraries in other languages.

5 Technical Advantages

High-level abstraction :

Finex expresses complex financial concepts and operations with simple syntax, hiding the mathematical and computational details from the user. It allows users to focus on “what” (the financial strategy) rather than “how” (the implementation details).

Instead of writing thousands of lines of C++/python code for Monte-Carlo or Black-Scholes pricing, user just writes it in a single line.

Type-Safety and error-reduction :

Finex restricts invalid operations outside the domain, reducing run-time errors. It uses strongly typed assets, enumerations, explicit models.

And the use of optimised solvers, memory safety enhance the performance of the code.

Declarative syntax :

The declarative syntax helps in describing the desired output instead of step-by-step execution. Finex helps user to just specify the statistics, not how to calculate them.

Unified platform :

Instead of using various workflows for various features, Finex is a unified platform to perform various operations in a single language.

Prior to Finex, traders backtest in Python, price derivatives in C++, stress test in R, report in Excel. Finex unifies all this various functionalities in one language.

Expressiveness and Extensibility :

Finex efficiently and concisely defines concepts of financial domain, making it more readable and programmable to the user. Use of **domain-specific vocabulary** like portfolio, asset, rebalance, scenario,... reduces the “semantic gap” between finance vocabulary and generic programming terms.

The design of **asset** class as an abstract class makes it extensible for future-purposes. The user defined scenario and rule blocks which can be subjected to changes enhance the extensibility

of language.

6 Comparison with Alternatives

Today’s financial workflow is fragmented. Traders use Python/C++ backtesting engines, risk teams rely on separate Monte Carlo tools, and regulators run stress scenarios in Excel or large data warehouses. Quants must also hand-pick the right solver for each instrument, code it themselves, and manage performance optimization—making the process slow, complex, and error-prone.

Our DSL offers a better path. It unifies all of these steps into one finance-native language where users simply describe scenarios in natural terms. The DSL then selects solvers and delivers outputs such as loss/gain expectations, value-at-risk, and expected shortfall. This eliminates duplication, reduces technical overhead, and creates a single framework that serves traders, risk managers, and regulators with equal consistency and speed.

While there are existing general purpose programming languages and finance DSLs, they either require extensive knowledge in programming and the field of finance (in case of general purpose languages) or are made for a particular sub-domain of finance. Our aim for this DSL is that a non-programmer can use the tools without knowing about their implementations. We aim to provide a layer of abstraction to the common user without them needing to have the required technical specialization.

7 Programming Principles

Performance:

The simple, high-level syntax acts as an interface to powerful, optimized financial modeling, where commands like `simulate` or `risk.contribution` trigger pre-compiled high-speed routines. This architecture enables implicit parallelism, allowing the engine to automatically distribute tasks like Monte Carlo simulations or scenario analyses across multiple CPU cores without user intervention.

Portability:

Our language ensures portability by allowing the same Finex scripts to run across diverse environments - from a local laptop to a cloud-based production system without modification. The scripts are platform-independent and will execute identically wherever the Finex runtime is installed. This is achieved through abstracted I/O commands like `Load` or `GenerateReport`, which separate the financial logic from the system’s specific file or API handling. By creating a self-contained ecosystem that integrates all necessary financial tools, Finex reduces external dependencies, making it simple to deploy and run in any environment.

Programmability:

The declarative syntax closely mirrors the language of finance, with commands like `add Stock("GOOG") to MyPortfolio` that allows professionals to focus on their strategy ("what") rather than the implementation ("how"). This is made possible through high-level abstractions such as `Portfolio`, `TimeSeries`, and `BlockScholes`, which encapsulate complexity behind simple terms. Finex unifies the entire workflow - from risk analysis and derivative pricing to time-series forecasting into a single language.

8 Code samples

```
1  @accounting
2  @statmodel
3  @containers
4  @datastream
5
6  void rebalance_portfolio(portfolio p modifiable) {
7      for(holding in p.assets) {
8          if(holding == assets.stock and holding.ticker in ["AAPL", "
          GOOGLE"]) {
9              if(holding.current_price() < holding.avg_cost() * 0.95) {
10                 print("Rebalancing: Buying more of ", holding.ticker);
11                 p.buy(asset.stock(holding.ticker, 10, holding.
                    current_price()));
12             }
13         }
14     }
15 }
16
17 list<double> analyze_portfolio(portfolio p) {
18     scenario_result = scenario analysis(p, scenarios: ["bull", "bear",
19         "flat", "inflation"]);
20     sensitivity_result = sensitivity analysis(p, asset.stock("AAPL"),
21         0.1);
22     monteCarloSim = Monte_Carlo(p, iterations: 10000);
23     portfolio_var = var(p, 0.99);
24     portfolio_cvar = cvar(p, 0.99);
25     portfolio_sharpe = sharperatio(p);
26
27     print("Scenario Analysis: ", scenario_result);
28     print("Sensitivity (AAPL): ", sensitivity_result);
29     print("Monte Carlo Result: ", monteCarloSim);
30     print("VaR: ", portfolio_var, " | CVaR: ", portfolio_cvar);
31     print("Sharpe Ratio: ", portfolio_sharpe);
32
33     return [scenario_result, sensitivity_result, monteCarloSim,
34         portfolio_var, portfolio_cvar, portfolio_sharpe];
35 }
36
37 void backtest_and_decide(portfolio p, string ticker) {
38     data = open file("historical_prices.csv");
39     df = dataframe(data);
40     filtered = filter(df, condition: "ticker == '" + ticker + "'");
41     backtest_result = backtest(p, filtered);
42
43     print("Backtest result for ", ticker, ": ", backtest_result);
44
45     if(backtest_result.pnl < 0) {
46         print("Decision: Backtest shows losses, reducing exposure to "
47             , ticker);
48         p.sell(asset.stock(ticker, 5, p.asset_price(ticker)));
49     } else {
```

```

46         print("Decision: Backtest positive, increasing exposure to ",
47               ticker);
48     }
49 }
50
51 int main() {
52     user Me ("John", "john@finex.com");
53     user Partner ("Lola", "lola@finex.com");
54
55     account myAccount (Me, 100000.0);
56     account partnerAccount (Partner, 50000.0);
57
58     Me.deposit(10000.0);
59     Me.transfer(partnerAccount, 3000.0);
60     Partner.withdraw(500.0);
61
62     portfolio myPortfolio (myAccount, [
63         asset.stock("AAPL", 50, 175.0),
64         asset.stock("GOOGL", 30, 2800.0),
65         asset.etf("SPY", 100, 450.0),
66         asset.bond("US10Y", 10, 1000.0)
67     ]);
68
69     portfolio partnerPortfolio (partnerAccount, [
70         asset.stock("TSLA", 20, 700.0),
71         asset.etf("QQQ", 50, 370.0)
72     ]);
73
74     rebalance_portfolio(myPortfolio);
75
76     results = analyze_portfolio(myPortfolio);
77
78     backtest_and_decide(myPortfolio, "AAPL");
79     backtest_and_decide(partnerPortfolio, "TSLA");
80
81     myPortfolio.buy(asset.option("AAPL", 180.0, "call", "2025-12-31"),
82                    5);
83
84     plot_graph(myPortfolio, "pie", "current_value");
85     plot_graph(myPortfolio, "line", "historical_value");
86
87     print("Final Portfolio Value: ", myPortfolio.curr_value());
88     return 0;
89 }

```

9 Precedence and Associativity

Rank	Operators	Description	Assoc.
1 (Highest)	:: @	Member Access (Label) Library Access	L-R
2	() [] ->	Function Call Subscript Member Access	L-R
3	+ - not (type)	Unary Plus Unary Minus Logical NOT Cast	R-L
4	^	Exponentiation	R-L
5	* / // %	Multiplication Real Division Integer Division Modulo	L-R
6	+ -	Addition Subtraction	L-R
7	> < >= <=	Greater Than Less Than Greater or Equal Less or Equal	L-R
8	== != ===	Equality Inequality Type Checking	L-R
9	in	Membership Operator	L-R
10	and	Logical AND	L-R
11	or	Logical OR	L-R
12	throw = += -= *= /= %= ^=	Throw operator Assignment Add and Assign Subtract and Assign Multiply and Assign Divide and Assign Modulo and Assign Power and Assign	R-L
13 (Lowest)	,	Comma	L-R